National University of Computer and Emerging Sciences
Islamabad Campus

FYP Proposal

Submitted by: Fahad Raza, Syed Inshaal Yousaf, Inshara Aman
Roll number: 22i-1768, 22i-7439, 22i-1653

Table of Contents

Project Title: 4
1. Problem Statement 4
2. Project Objectives 4
3. Project Scope 5
4. Methodology 5
Phase 1 — Research & Design 6
Phase 2 — Compliance Engine Development 6
Phase 3 — Application & Pipeline Security Integration 6
Phase 4 — Dashboard & Visualization 6
Phase 5 — Reporting & Alerts 6
5. Web Dashboard 7
1. GRC-First Control Mapping & Compliance 7
2. Security Data Integration 7
3. Automated Threat Modeling 7
4. Secure Architecture Recommendations 7
5. Code-Level Secure Coding Advisor 7
6. Attack & Vulnerability Heatmaps 7
7. Remediation & Continuous Verification 8
6. Tools and Technologies 8
6.1 Application Domain 8
6.2 Pipeline and Tooling Domain (Core DevSecOps Components) 9
6.3 Governance, Risk & Compliance (GRC) Dashboard 10
6.4 Diagram 10
6.5 Table 11
7. GRC Frameworks 12
7.1 NIST Cybersecurity Framework (NIST CSF) 12
7.2 CIS Controls v8 (Technical Hardening Baseline) 13

7.3 ISO/IEC 27001 (Selective Mapping) 13
7.4 NIST SP 800-218 (Secure Software Development Framework — SSDF) 14
7.5 ISO/IEC 27034 (Application Security Controls) 14
8. End-to-End Working Model 15
9. Project Deliverables 16
10. Comparison with Existing Academic, Open-Source, and Commercial Projects 16

Project Title:

Automated Governance, Risk, and Compliance (GRC) Framework and Policy Enforcement System for DevSecOps Environment

Domains: GRC and DevSecOps

1. Problem Statement

Modern software development teams struggle to maintain consistent security and compliance due to rapid release cycles, fragmented tools, and heavy reliance on manual reviews. Traditional audit-driven approaches fail to keep pace with agile development, resulting in compliance gaps, undocumented risks, and inconsistent enforcement of organizational policies.

This Final Year Project aims to address these challenges by designing and implementing an automated Governance, Risk, and Compliance (GRC) framework embedded directly into a DevSecOps pipeline for surface-web applications. The solution will unify code-quality analysis, vulnerability scanning, dependency checks, container security validation, and policy enforcement through a centralized, intelligent governance dashboard. By embedding compliance checks into CI/CD processes and applying "Policy-as-Code," the system will enable continuous enforcement of standards such as NIST CSF, CIS Controls, and ISO 27001—eliminating manual effort and ensuring reliable, real-time compliance across development workflows.

2. Project Objectives

1. **Design a unified Governance, Risk, and Compliance (GRC) framework** tailored for DevSecOps environments and aligned with industry standards such as NIST CSF, ISO 27001, and CIS Benchmarks.
2. **Develop a "Compliance-as-Code" engine** capable of automatically evaluating application code, configuration files, dependency manifests, Dockerfiles, and CI/CD pipeline scripts against defined security and compliance policies.
3. **Automate risk assessment** using a structured rule-based scoring model, with optional machine-learning enhancements to prioritize high-impact violations dynamically.
4. **Integrate automated compliance checks into CI/CD tools** such as GitHub Actions, Jenkins, or GitLab CI to enforce continuous security and policy validation during every build and deployment stage.

5. **Develop a real-time GRC dashboard** that visualizes compliance scores, violation details, severity levels, and recommended remediation actions for developers and security teams.
6. **Generate audit-ready compliance reports** that map all identified issues to relevant control frameworks (ISO 27001 Annex A, NIST CSF categories, CIS controls), enabling standardized and defensible audit evidence.

3. Project Scope

In-Scope Features	Description
Policy Definition Framework	Develop a structured YAML/JSON-based policy format for secure coding standards, dependency rules, CI/CD governance checks, and pipeline-level compliance requirements.
Automated Compliance Engine	Build an engine that scans application configuration files—such as Dockerfiles, package manifests, environment variables, and CI/CD YAML—to detect violations of defined security and operational policies.
CI/CD Security Integration	Integrate automated DevSecOps checks into GitHub Actions or Jenkins, including SAST (SonarQube), SCA (Snyk), container scanning (Trivy), and policy-as-code enforcement before deployment.
Source Code & Dependency Security	Enable automated scanning of source code, third-party libraries, and container images to identify vulnerabilities, weak coding patterns, and risky dependencies.
Compliance & Security Dashboard	Develop a web-based dashboard displaying compliance status, policy violations, risk scores, pipeline security events, and historical trends from pipeline runs.
Real-Time Alerting System	Implement instant alerts via email/Slack whenever high-severity pipeline violations, build failures caused by security checks, or policy breaches occur.

4. Methodology

The project will follow a modular and iterative development approach, ensuring each component is designed, tested, and integrated systematically.

Phase 1 — Research & Design

- Study the DevSecOps lifecycle, secure SDLC principles, and GRC frameworks (ISO 27001, NIST, CIS).
- Design the data model for policy representation based on the "Policy-as-Code" concept.
- Identify and evaluate relevant open-source security scanners (e.g., Checkov, OPA, Semgrep).

Phase 2 — Compliance Engine Development

- Implement the automated policy parser and violation-detection engine.
- Develop connectors for Dockerfiles, application manifests (package.json, requirements.txt), and CI/CD pipeline scripts (GitHub Actions/Jenkins YAML).
- Define severity levels and calculate compliance scores for each pipeline execution.

Phase 3 — Application & Pipeline Security Integration

- Embed SAST, SCA, and container-security tools (SonarQube, Snyk, Trivy) directly into the CI/CD pipeline.
- Implement automated checks for insecure coding patterns, weak dependencies, container misconfigurations, and unsafe deployment practices.
- Establish enforcement logic to block builds or require approvals when critical violations occur.

Phase 4 — Dashboard & Visualization

- Build a web interface using Flask + React.
- Implement compliance heatmaps, trend graphs, severity analytics, and risk categorization.
- Provide drill-down views showing violations at code, dependency, and pipeline levels.

Phase 5 — Reporting & Alerts

- Generate detailed compliance reports mapped to NIST, ISO 27001, and CIS benchmarks.
- Configure a real-time alerting system (Slack, Email) for critical non-compliance, policy breaches, and high-risk pipeline failures.

5. Web Dashboard

The Web Dashboard serves as an **intelligent GRC-driven DevSecOps security layer**, unifying governance, threat modeling, architecture evaluation, secure coding review, and continuous verification.

1. GRC-First Control Mapping & Compliance

The dashboard maps all development artefacts, scan results, and architectural data to GRC frameworks (NIST CSF, ISO 27001, SP 800-218, ISO 27034).
It immediately highlights **missing, weak, or partially implemented controls** and produces a concise **"You lack here"** compliance gap matrix.

2. Security Data Integration

It aggregates outputs from integrated DevSecOps tools—including SonarQube (SAST), Snyk (SCA), Trivy (container/IaC scanning), and Anchore (policy enforcement)—together with GitHub codebase data and service architecture metadata to form a unified security view..

3. Automated Threat Modeling

Based on the system's structure, the dashboard automatically applies:

- **STRIDE** for technical threats
- **PASTA** for business-risk simulation
- **LINDDUN** for privacy threats
- **Attack Trees** for adversarial paths

4. Secure Architecture Recommendations

Using secure architecture principles and proven design patterns, it suggests improvements related to least privilege, secure defaults, encryption, API gateway protection, Zero Trust, and defense-in-depth.

5. Code-Level Secure Coding Advisor

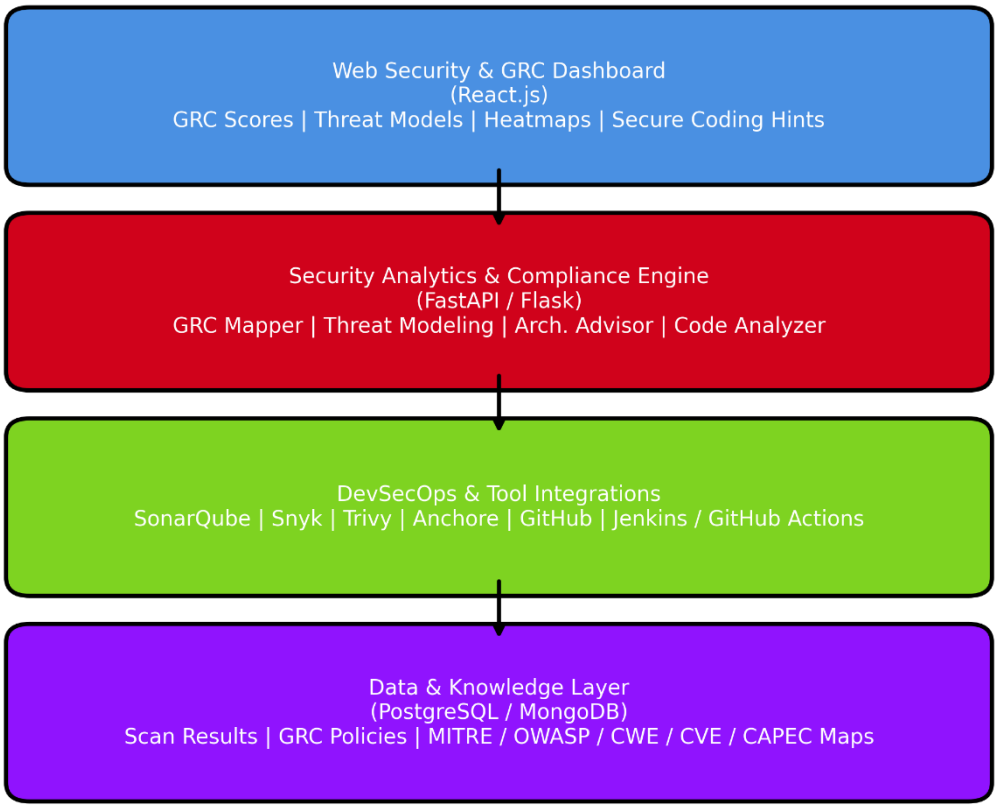
The dashboard analyzes the GitHub codebase (independent of other tools) to detect injection flaws, broken authentication, hardcoded secrets, insecure APIs, missing encryption, and improper error handling—then recommends precise fixes.

6. Attack & Vulnerability Heatmaps

All findings are mapped to **MITRE ATT&CK**, **OWASP Top 10/API Top 10**, **CWE software weaknesses**, **CVE library vulnerabilities**, and **CAPEC attack patterns**, generating clear heatmaps that highlight exposed attack paths, underlying weaknesses, and exploitation severity.

7. Remediation & Continuous Verification

For each issue, the dashboard recommends actionable remediation, explains potential impact, and re-scores compliance, architecture health, and threat exposure after every pipeline run.



6. Tools and Technologies

6.1 Application Domain

Primary Focus: Surface Web Application + REST APIs

A simple demonstrative web application will be developed using a mainstream framework such as **Node.js (Express)**, **Django**, or **Spring Boot**. The application will serve as the *test system* for validating DevSecOps and GRC capabilities. It will be used to:

- Push code to GitHub/GitLab repositories
- Trigger automated CI/CD pipelines
- Run code, dependency, and container security scans
- Generate machine-readable security reports for the GRC engine

This ensures a controlled environment for demonstrating continuous compliance across a real development workflow.

6.2 Pipeline and Tooling Domain (Core DevSecOps Components)

The project uses a full DevSecOps pipeline incorporating industry-standard tools.

1. Source Code Management (SCM)

- GitHub or GitLab repositories
- Branch protections, pull-request checks, enforced code review rules
- Required security and quality status checks

2. CI/CD Pipeline Orchestration

Using **GitHub Actions**, **GitLab CI**, or **Jenkins** (one platform selected for the final system). Pipeline stages will include:

- **Build** → compilation / container build
- **Test** → unit tests or basic functional checks
- **Scan** → automated SAST, SCA, and container security
- **Deploy** → deploy to staging/test environments (Docker-based)

3. Code & Dependency Security

- **SonarQube** → code quality, code smells, and basic SAST
- **Snyk** → Software Composition Analysis (SCA) for dependency vulnerabilities

4. Container / Image Security

- **Trivy** → container image vulnerability scanning



- **Anchore (optional)** → policy-based scanning and GRC alignment for container images

Final DevSecOps Scope:
SCM → CI/CD → Code Scanning → Dependency Scanning → Container Scanning → Optional IaC checks.

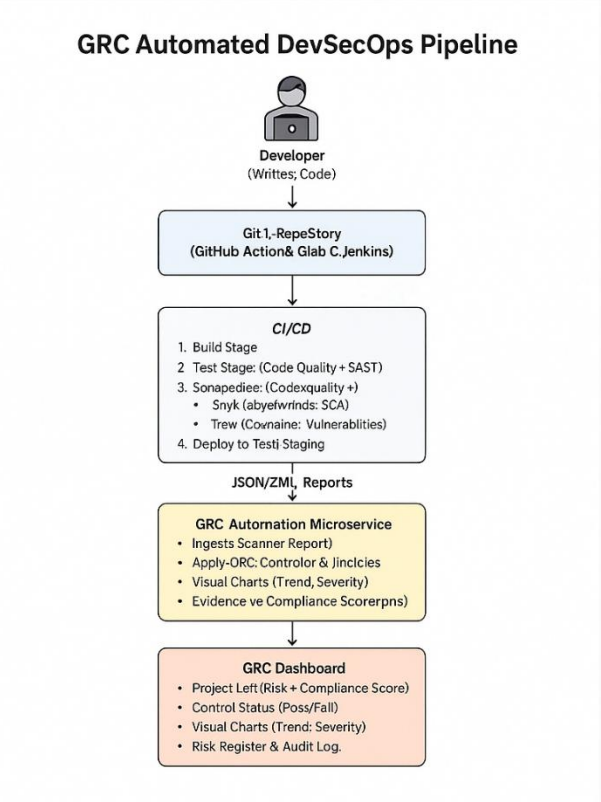
6.3 Governance, Risk & Compliance (GRC) Dashboard

This forms the core *innovation* of the FYP. The dashboard is a surface-web governance tool that:

- Aggregates security scan outputs from SonarQube, Trivy, Snyk, Anchore
- Maps each finding to compliance controls (NIST CSF, CIS Controls, ISO 27001)
- Calculates and displays compliance scores
- Shows pipeline-level risk posture for each project
- Maintains a simple risk register and audit trail

This is the "Compliance Automation Layer" that converts raw DevSecOps scanning data into meaningful GRC insights.

6.4 Diagram



GRC Mapping Overview

DevSccOp=Tool	Purpose	Mapped Curttrols (MIST CSR)	Mapped Controls (CIS-iD)	Mapped Controls IISO,00,Of: Antiwase: (i1 218 Annnor A
SonarQube	Code quality. SAST for Vulnerabilities	PP.IP-1.3	CIS 16 (Application Softwoni Security)	A12.28 (Secterr.Coding) A.At Isvconowtty tonsinght Bewlpou
Snyk	Software Conunittition Analysis (SCA)	DE. CM-8	CIS 7 (Confiriciaus VulnenBy.ilis, Management	A 16.61 (Technical Vutnenability Management)
Trivy	Container Vulnerability Scerroanig	PR.IP-12	CIS 4 Sacsuma Configurations)	A.11.3.8 (Secure Systerty Testting)
Anchore	Container polici/enforcement	PR,IP7-3	CIS.2 (Sex IIB)	A 16.2.5 (Secure System Phglheering Principles)
GltHub/Gditab SCM	Version control, (nsnoir protection	CIS.4	— CIS.6	A.6.3 (User Acces Manageememen.)
CI/CD Pipeline	Automated build.deilt, deploy	ID.AM. 1, P.IP.5	CIS.4 C.IS 4, 2.IP.IP.17	A 12.14 (Septuntion of Dev/Test /Piod)
Docker / Kubernates	Automated build. test, deplay	CIICD. Droscetted	PR. IP -4 RS.Att- 7	A 17.5 (Change Control)
Dacker / Kubernetes	Container runt'me: / oranstraction	A12 1.PR Ji T	A. 11 2 PR,PM.3	A.12.5 (Vimerability) Management)

6.5 Table

Tool	Purpose	NIST CSF Mapping	CIS Controls Mapping	ISO 27001 Mapping
SonarQube	Code quality analysis and Static Application Security Testing (SAST)	PR.IP-1 (Secure development), PR.DS-6 (Application security integrity)	CIS Control 16 – Application Software Security	A.8.28 (Secure coding), A.8.29 (Security testing)
Snyk	Software Composition Analysis (SCA) for third-party dependencies	DE.CM-8 (Vulnerability monitoring)	CIS Control 7 – Continuous Vulnerability Management	A.12.6.1 (Technical vulnerability management)
Trivy	Container image vulnerability scanning	PR.IP-12 (Secure configuration management)	CIS Controls 4 & 7 – Secure Configuration;	A.14.2.8 (System security testing)

Tool	Purpose	NIST CSF Mapping	CIS Controls Mapping	ISO 27001 Mapping
			Vulnerability Management	
Anchore	Policy-based container image enforcement	PR.IP-3 (Configuration enforcement)	CIS Control 2 – Software Asset Management	A.14.2.5 (Secure configuration procedures)
GitHub / GitLab	Source code management, branch protection, access control	ID.AM-1 (Asset identification)	CIS Controls 2 & 6 – Inventory; Access Control	A.9.2 (User access management), A.12.1.2 (Change management)
CI/CD Pipeline	Automated build, test, scan, and deployment workflow	PR.IP-3 (Configuration management), RS.MI-3 (Mitigation)	CIS Controls 4 & 16 – Secure Configuration; Application Security	A.12.1.4 (Separation of environments), A.12.5 (Secure change control)
Docker / Kubernetes (Local)	Container runtime and orchestration	DE.CM-7 (Security monitoring)	CIS Controls 4 & 11 – Secure Configuration; Container Security	A.13.1.1 (Network security controls)

7. GRC Frameworks

7.1 NIST Cybersecurity Framework (NIST CSF)

NIST CSF serves as the primary conceptual baseline. Only the functions relevant to DevSecOps will be used:

NIST Function	Relevant Categories	Application in FYP
Identify (ID)	Asset Management	Repositories, pipelines, artifacts
Protect (PR)	Access Control, Data Security, Protective Technology	Branch protection, secrets management, integrated security tools
Detect (DE)	Continuous Monitoring	Regular SAST, SCA, container scans

NIST Function	Relevant Categories	Application in FYP
Respond (RS)	Mitigation	Remediation and rescan workflows

Example Mapping:

- **Policy:** "All container images must be scanned before deployment."
- **NIST CSF:** PR.IP-12
- **Implementation:** Trivy scan in CI pipeline
- **Evidence:** JSON scan report displayed in GRC dashboard

7.2 CIS Controls v8 (Technical Hardening Baseline)

CIS v8 provides practical, technical guidance directly applicable to DevSecOps. Key controls selected include:

- **CIS Control 2** — Inventory of Software Assets (Repositories, pipelines)
- **CIS Control 4** — Secure Configuration of Software
- **CIS Control 7** — Continuous Vulnerability Management
  - Implemented via: Snyk + Trivy + Anchore
- **CIS Control 16** — Application Software Security
  - Implemented via: SonarQube (SAST checks)

These controls will appear as compliance indicators within the dashboard.

7.3 ISO/IEC 27001 (Selective Mapping)

Full ISO 27001 implementation is beyond FYP scope; instead, a light-weight mapping to selected Annex A controls will be provided:

- **A.5 — Information Security Policies**
- **A.8.28 — Secure Coding**
- **A.8.29 — Security Testing and Acceptance**

The GRC dashboard will show how pipeline policies support ISO controls, enhancing the project's industry relevance.

7.4 NIST SP 800-218 (Secure Software Development Framework — SSDF)

NIST SP 800-218 provides a practical, prescriptive baseline for integrating security into every stage of DevSecOps. The FYP adopts the SSDF practices most relevant to CI/CD, secure coding, and automated governance.

Relevant Practice Groups and FYP Application

SSDF Practice	Focus Area	Application in FYP
PO – Prepare the Organization	Define roles, policies, governance	GRC policy definitions and compliance scoring
PS – Protect Software	Secure code, dependencies, secrets	Snyk (CVE checks), Trivy (images/IaC), secret-detection
PW – Produce Well-Secured Software	Threat modeling + SAST	STRIDE/PASTA engine, SonarQube scanning, architecture advisor
RV – Respond to Vulnerabilities	Triage + remediation	Automated remediation guidance + rescan workflow

Example Mapping

- **Policy:** "All dependencies must be checked against known CVEs."
- **SP 800-218:** PW.4 / PS.3
- **Implementation:** Snyk dependency scanning
- **Evidence:** CVE report + remediation suggestions in dashboard

SP 800-218 strengthens the project's credibility by aligning it with secure software supply chain requirements used by major enterprises and U.S. federal agencies.

7.5 ISO/IEC 27034 (Application Security Controls)

ISO 27034 provides internationally recognized guidance for secure application design, threat-aware architecture, and secure coding—directly matching core features of the FYP.

Relevant Domains and FYP Application

ISO 27034 Area	Focus Area	Application in FYP
ASM — Application Security Management	Security governance for applications	GRC control mapping and policy verification
Application Security Controls (ASCs)	Auth, access control, validation, crypto	Architecture advisor + secure coding rule engine
Secure SDLC Integration	Threat modeling + verification	STRIDE/PASTA outputs connected to CI/CD controls
Developer Security Guidelines	Coding rules and secure practices	Code-level analysis with OWASP/CWE/CAPEC mapping

Example Mapping

- **Control:** Input Validation (ASC)
- **ISO 27034:** Application Security Controls Domain
- **Implementation:** SonarQube + custom secure coding checks
- **Evidence:** Line-level findings with recommended fixes

ISO 27034 adds architectural and code-level depth to the dashboard, ensuring that secure design, secure coding, and threat modeling are aligned with global application security standards.

8. End-to-End Working Model

1. **Developer commits code** → GitHub/GitLab.
2. **CI/CD pipeline triggers:**
    - Build
    - Unit tests
    - SonarQube scan
    - Snyk scan
    - Trivy/Anchore container scan
3. Each tool produces **machine-readable JSON/XML reports**.
4. **GRC Automation Service:**
    - Ingests scan data
    - Applies “Policy-as-Code” rules



- Maps findings to NIST/CIS/ISO controls
- Calculates risk and compliance scores

5. **Surface-Web Dashboard:**
   - Shows overall compliance score
   - Highlights violations
   - Tracks trends and pipeline history
   - Maintains a simple risk register

GRC Frameworks Implemented

- **NIST CSF** — primary governance framework
- **CIS Controls v8** — technical security baseline
- **ISO 27001 Annex A (selected controls)** — light mapping

9. Project Deliverables

1. **Functional Web-Based GRC Compliance Dashboard**
   A fully developed dashboard providing visibility into policy violations, risk scores, and compliance trends.
2. **Operational Policy Engine ("Policy-as-Code")**
   A rule-based engine capable of parsing YAML/JSON policies, enforcing controls, and detecting violations across application and pipeline configurations.
3. **CI/CD Integration Demonstration**
   End-to-end proof of integration with at least one CI/CD platform (GitHub Actions or Jenkins), including SAST, SCA, and container-security checks.
4. **Application & Pipeline Security Scanning Module** *(Replaces cloud scanning)*
   A module that performs automated scanning of Dockerfiles, dependencies, configuration files, and CI/CD workflows to detect insecure patterns, weak dependencies, and misconfigurations.
5. **Auto-Generated Risk & Compliance Report (ISO/NIST Mapped)**
   A structured report generated after each pipeline run, highlighting violations, severity levels, and alignment with ISO 27001, NIST controls, and CIS benchmarks.
6. **Complete Project Documentation and Presentation Package**
   Comprehensive documentation covering architecture, methodology, configuration, user guide, and a final presentation summarizing the system.

10. Comparison with Existing Academic, Open-Source, and Commercial Projects

This section compares the proposed FYP with existing academic theses, open-source tools, and commercial products that aim to integrate DevSecOps practices with Governance, Risk, and

Compliance (GRC) frameworks. The comparison highlights how the proposed solution differs in terms of integration depth, GRC alignment, and innovation.

The following table summarizes the distinguishing aspects of the proposed FYP:

Project/Product	Category	Integrated Tools	GRC Frameworks	Key Features	Limitations / Gaps
DevSecOps–ISO 27001 Thesis	Academic	Conceptual CI/CD tools	ISO 27001	Mapped ISO controls to DevOps processes	No implementation; manual mapping only
Solanke Paper (2022)	Academic	General SAST/DAST	NIST CSF	Policy-as-code, compliance monitoring	Theoretical model, no pipeline/tool
GovReady-Q	Open-Source	None (integrates via OSCAL)	NIST 800-53, FedRAMP	Automated compliance documentation	Not a scanner, requires manual integration
OpenSCAP	Open-Source	Config scanners	CIS Benchmarks, NIST 800-53	Host/container compliance audits	Limited app code support, no dashboard
Prowler	Open-Source	Cloud API scanners	CIS, NIST CSF, ISO 27001 (cloud)	Cloud audit reports mapped to controls	No code/pipeline integration
Aptori	Commercial	Built-in SAST/SCA, CI integrations	NIST CSF, PCI, HIPAA	AI vulnerability detection with control mapping	Platform-locked, limited audit trail
Drata	Commercial	Snyk, Qualys, etc.	SOC 2, ISO 27001, NIST	Evidence collection, control dashboards	Not pipeline-native, depends on external scanners

GitLab Ultimate	Commercial	GitLab SAST/DAST	ISO 27001, CIS, SOC 2	Built-in compliance enforcement in CI	GitLab-only, limited flexibility
User's FYP Project	Custom FYP	SonarQube, Snyk, Trivy, Anchore	NIST CSF, CIS v8, ISO 27001	Full-stack integration with dashboard, audit log, risk register	Novel, end-to-end approach combining security and governance

As seen in the table above, most academic and open-source solutions either provide theoretical models or partial implementations lacking end-to-end integration. Commercial platforms often focus on either security scanning or compliance reporting but rarely both in an open, developer-oriented fashion. The proposed FYP stands out by integrating multiple security tools (SonarQube, Snyk, Trivy, Anchore) into a DevSecOps pipeline and mapping their outputs directly to NIST CSF, CIS v8, and ISO 27001 frameworks. Additionally, it introduces a GRC automation microservice and a governance dashboard complete with audit logs and risk registers, making it a comprehensive and innovative solution.